

The HP 2D-Protein Folding problem models a simplified version of a well-known problem in computational biology. It consists in finding the spatial conformation of a protein (i.e., a sequence of amino-acids) with minimal energy.

The simplifications with respect to the real problem are twofold.

First, the 20-letter alphabet of amino-acids is reduced to a two-letter alphabet, namely H and P. H represents hydrophobic amino-acids, whereas P represents polar or hydrophilic amino-acids.

Second, the conformation of the protein is limited to a bi-dimensional discrete space.

Nonetheless, these limitations have been proven to be very useful for attacking the whole protein conformation prediction problem. The simplified version is known to be NP-complete [?] and very hard to solve in practice.

Given the sequence of amino-acids of the protein, i.e., a string over {H,P} of length n , the problem aims to find a connected shape for it on a 2D grid (with coordinates in $[-(n-1), (n-1)]$, starting at a pre-determined position, e.g., $(0, 0)$), which is non-crossing, and such that the number of “contacts”, i.e., the number of non-sequential pairs of H's for which the Euclidean distance of the positions is 1 is maximized (the overall energy is the opposite of the number of contacts). Figure 1.1 shows a possible conformation of the protein “PHHPHPPHP”, with overall energy -2 .

The goal of this exercise is to define pseudo-code (and possibly an implementation in any high-level language) of a portfolio of programs that, given an instance of the problem (namely, a value for positive integer n and the linear structure of a protein in the form of a string over symbols ‘H’ and ‘P’) find an optimal 2D conformation of the input protein. Each program in the portfolio shall use a solving method inspired by a non-informed/informed search state exploration algorithm shown in the course, and in particular:

1. depth first search
2. breadth first search
3. min cost search
4. iterative deepening search
5. best-first greedy search
6. A*

In case of informed search algorithms, a suitable heuristic function shall be designed.

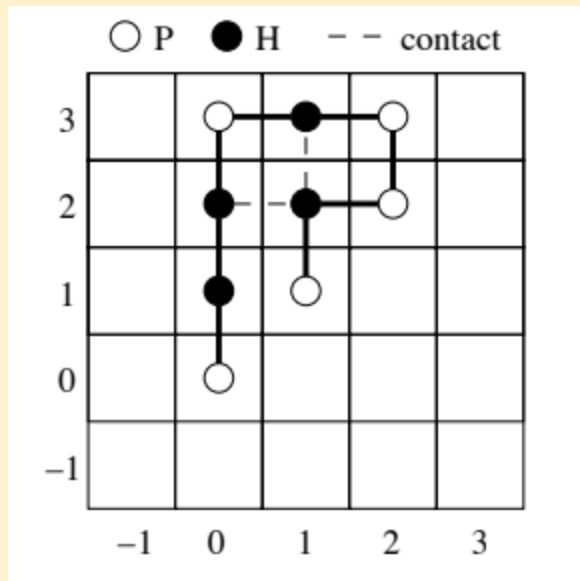


Figure 1.1: HP 2D-Protein folding problem: A possible conformation for protein “PHHPHPPHP” with two contacts, and overall energy -2 .

Per applicare gli algoritmi di ricerca dobbiamo trasformarlo in un problema di ricerca. Assegniamo agli stati la rappresentazione di una conformazione parziale della proteina. All'inizio abbiamo solo il primo amminoacido, fissato nella posizione iniziale, ad esempio $P1 = (0,0)$. Uno stato completo contiene le coordinate di tutti gli amminoacidi. Ad ogni passo dobbiamo scegliere dove posizionare l'amminoacido. Se l'ultimo amminoacido si trova in (x,y) , puoi provare a posizionare il successivo in una delle quattro posizioni: $(x+1,y)$, $(x-1,y)$, $(x,y+1)$, $(x,y-1)$. Uno stato è goal se tutti gli amminoacidi della sequenza sono stati posizionati e se la conformazione rispetta i vincoli. Per trovare lo stato goal ottimo dobbiamo confrontare le conformazioni complete e selezionare quella con il maggior numero di contatti.

DFS

```
DFS(proteina):
    protLen = lunghezza(proteina)
    limite = protLen - 1
    stato_iniziale:
        posizioni = [(0,0)]
```

```

    contatti = 0
coda = [stato_iniziale]
miglior_stato = null
max_contatti = -1
while coda non è vuota:
    stato = estrai_primo(coda)
    currStateProtLen = lunghezza(stato.posizioni)
    # Se tutti gli amminoacidi sono stati posizionati
    if currStateProtLen == protLen:
        if stato.contatti > max_contatti:
            max_contatti = stato.contatti
            miglior_stato = stato
        continua
    # Posizione del ultimo amminoacido
    (x, y) = ultimo(stato.posizioni)
    # Genera le 4 posizioni adiacenti
    vicini = [
        (x+1, y),
        (x-1, y),
        (x, y+1),
        (x, y-1)
    ]
    for ogni posizione (nx, ny) in vicini:
        # Controlla i limiti della griglia
        if nx < -limite or nx > limite:
            continua
        if ny < -limite or ny > limite:
            continua
        # Controlla che la posizione sia libera
        if (nx, ny) è già in stato.posizioni:
            continua
        # Calcola i nuovi contatti

```

```

        nuovi_contatti = 0
        if proteina[currStateProtLen] == 'H':
            for j da 0 a currStateProtLen-2:
                if proteina[j] == 'H' &&
                    (nx, ny) e stato.posizioni[j] sono
adiacenti:
                            nuovi_contatti += 1
        # Crea il nuovo stato
        newStato.posizioni =
            stato.posizioni + [(nx, ny)]

        newStato.contatti =
            stato.contatti + nuovi_contatti

        # Inserisce in testa alla coda
        inserisci_in_testa(coda, newStato)
    return miglior_stato, max_contatti

```

BFS

```

BFS(proteina):
    protLen = lunghezza(proteina)
    limite = protLen - 1
    stato_iniziale:
        posizioni = [(0,0)]
        contatti = 0
    coda = [stato_iniziale]
    miglior_stato = null
    max_contatti = -1
    while coda non è vuota:
        stato = estrai_primo(coda)

```

```

currStateProtLen = lunghezza(stato.posizioni)
# Se tutti gli amminoacidi sono stati posizionati
if currStateProtLen == protLen:
    if stato.contatti > max_contatti:
        max_contatti = stato.contatti
        miglior_stato = stato
    continua
# Posizione del ultimo amminoacido
(x, y) = ultimo(stato.posizioni)
# Genera le 4 posizioni adiacenti
vicini = [
    (x+1, y),
    (x-1, y),
    (x, y+1),
    (x, y-1)
]
for ogni posizione (nx, ny) in vicini:
    # Controlla i limiti della griglia
    if nx < -limite or nx > limite:
        continua
    if ny < -limite or ny > limite:
        continua
    # Controlla che la posizione sia libera
    if (nx, ny) è già in stato.posizioni:
        continua
    # Calcola i nuovi contatti
    nuovi_contatti = 0
    if proteina[currStateProtLen] == 'H':
        for j da 0 a currStateProtLen-2:
            if proteina[j] == 'H' &&
                (nx, ny) e stato.posizioni[j] sono
adiacenti:

```

```

        nuovi_contatti += 1

    # Crea il nuovo stato
    newStato.posizioni =
        stato.posizioni + [(nx, ny)]

    newStato.contatti =
        stato.contatti + nuovi_contatti

    # Inserisce in fondo alla coda
    inserisci_in_fondo(coda, newStato)
return miglior_stato, max_contatti

```

MinCost

```

MinCost(proteina):
    protLen = lunghezza(proteina)
    limite = protLen - 1
    coda = coda di priorità di stati ordinata per -(stato.contatti)
crescenti, inizialmente vuota;
    coda.add({
        posizioni = [(0,0)]
        contatti = 0
    })
    miglior_stato = null
    max_contatti = -1
    while coda non è vuota:
        stato = coda.estrai_primo()
        currStateProtLen = lunghezza(stato.posizioni)
        # Se tutti gli amminoacidi sono stati posizionati
        if currStateProtLen == protLen:
            if stato.contatti > max_contatti:
                max_contatti = stato.contatti
                miglior_stato = stato
            continua
        # Posizione del ultimo amminoacido
        (x, y) = ultimo(stato.posizioni)
        # Genera le 4 posizioni adiacenti

```

```

vicini = [
    (x+1, y),
    (x-1, y),
    (x, y+1),
    (x, y-1)
]
for ogni posizione (nx, ny) in vicini:
    # Controlla i limiti della griglia
    if nx < -limite or nx > limite:
        continua
    if ny < -limite or ny > limite:
        continua
    # Controlla che la posizione sia libera
    if (nx, ny) è già in stato.posizioni:
        continua
    # Calcola i nuovi contatti
    nuovi_contatti = 0
    if proteina[currStateProtLen] == 'H':
        for j da 0 a currStateProtLen-2:
            if proteina[j] == 'H' &&
                (nx, ny) e stato.posizioni[j] sono adiacenti:
                nuovi_contatti += 1
    # Crea il nuovo stato
    newStato.posizioni =
        stato.posizioni + [(nx, ny)]

    newStato.contatti =
        stato.contatti + nuovi_contatti

    # Inserisce nella coda nella posizione adatta
    coda.add(newStato)
return miglior_stato, max_contatti

```

Iterative deepening search

IDS non ha senso nel caso specifico dell'esercizio. IDS si compone di DFS che, ad ogni iterazione, scendono più in "profondità", esplorando nodi sempre più lontani dalla radice. Ma in

questo esercizio le soluzioni hanno tutte profondità $n-1$, quindi tutte le iterazione $m < (n-1)$ non troveranno mai una soluzione, il che riduce IDS ad una semplice DFS.

Best-first greedy search

Costruiamo l'euristica $h(n)$.

Innanzitutto dobbiamo trovare l'ottimo, e per farlo notiamo che data n la lunghezza della proteina, il massimo numero di contatti che può avere è $(k - 1)$ dove k è quel numero tale che $2 \cdot k = n$. Questo ovviamente assumendo che siano tutti aminoacidi H. Esempi:

- $n = 2 = 2 \cdot 1 \Rightarrow k = 1 \Rightarrow$ numero massimo di contatti è 0
- $n = 4 = 2 \cdot 2 \Rightarrow k = 2 \Rightarrow$ numero massimo di contatti è 1
- $n = 8 = 2 \cdot 4 \Rightarrow k = 3 \Rightarrow$ numero massimo di contatti è 2

Avendo ottenuto questo numero, possiamo usarlo per definire l'euristica

$$h(n) = (n_{\text{MassimoContatti}} - \text{contatti}) \cdot (n - \text{currStateProtLen})$$

dove contatti è il numero di contatti dello stato corrente, e currStateProtLen è il numero di aminoacidi già posizionati nello stato corrente. Prendiamo ad esempio $n = 10$, e notiamo che il suo numero massimo di contatti è 4:

- Passo 1 $\rightarrow h(n) = (4 - 0) \times (10 - 1) = 4 \times 9 = 36$ # non può avere contatti in questo passo
- Passo 2 $\rightarrow h(n) = (4 - 0) \times (10 - 2) = 4 \times 8 = 32$ # non può avere contatti in questo passo
- ...
- Passo 4
 - $\rightarrow h(n) = (4 - 0) \times (10 - 4) = 4 \times 6 = 24$ # se NON trova un contatto
 - $\rightarrow h(n) = (4 - 1) \times (10 - 4) = 3 \times 6 = 18$ # se trova un contatto

```
BFGS(proteina):
    protLen = lunghezza(proteina)
    nMassimoContatti = (k - 1) dove k è quel numero tale che 2k = protLen.
    limite = protLen - 1
    coda = coda di priorità di stati ordinata per stato.euristica
    crescenti, inizialmente vuota;
    coda.add({
        posizioni = [(0,0)]
        contatti = 0
        euristica = (nMassimoContatti - 0) x (protLen - 1)
    })
    miglior_stato = null
    max_contatti = -1
```

```

while coda non è vuota:
    stato = coda.estrai_primo()
    currStateProtLen = lunghezza(stato.posizioni)
    # Se tutti gli amminoacidi sono stati posizionati
    if currStateProtLen == protLen:
        if stato.contatti > max_contatti:
            max_contatti = stato.contatti
            miglior_stato = stato
        continua
    # Posizione del ultimo amminoacido
    (x, y) = ultimo(stato.posizioni)
    # Genera le 4 posizioni adiacenti
    vicini = [
        (x+1, y),
        (x-1, y),
        (x, y+1),
        (x, y-1)
    ]
    for ogni posizione (nx, ny) in vicini:
        # Controlla i limiti della griglia
        if nx < -limite or nx > limite:
            continua
        if ny < -limite or ny > limite:
            continua
        # Controlla che la posizione sia libera
        if (nx, ny) è già in stato.posizioni:
            continua
        # Calcola i nuovi contatti
        nuovi_contatti = 0
        if proteina[currStateProtLen] == 'H':
            for j da 0 a currStateProtLen-2:
                if proteina[j] == 'H' &&
                    (nx, ny) e stato.posizioni[j] sono adiacenti:
                    nuovi_contatti += 1
        # Crea il nuovo stato
        newStato.posizioni =
            stato.posizioni + [(nx, ny)]

        newStato.contatti =
            stato.contatti + nuovi_contatti

```

```

        newStato.euristica =
            (nMassimoContatti - newStato.contatti) x (protLen -
lunghezza(newStato.posizioni))

        # Inserisce nella coda nella posizione adatta
        coda.add(newStato)
    return miglior_stato, max_contatti

```

A*

```

A(proteina):
    protLen = lunghezza(proteina)
    nMassimoContatti = (k - 1) dove k è quel numero tale che 2k = protLen.
    limite = protLen - 1
    coda = coda di priorità di stati ordinata per -(stato.contatti) +
stato.euristica crescenti, inizialmente vuota;
    coda.add({
        posizioni = [(0,0)]
        contatti = 0
        euristica = (nMassimoContatti - 0) x (protLen - 1)
    })
    miglior_stato = null
    max_contatti = -1
    while coda non è vuota:
        stato = coda.estrai_primo()
        currStateProtLen = lunghezza(stato.posizioni)
        # Se tutti gli amminoacidi sono stati posizionati
        if currStateProtLen == protLen:
            if stato.contatti > max_contatti:
                max_contatti = stato.contatti
                miglior_stato = stato
            continua
        # Posizione del ultimo amminoacido
        (x, y) = ultimo(stato.posizioni)
        # Genera le 4 posizioni adiacenti
        vicini = [
            (x+1, y),
            (x-1, y),
            (x, y+1),

```

```

        (x, y-1)
    ]
    for ogni posizione (nx, ny) in vicini:
        # Controlla i limiti della griglia
        if nx < -limite or nx > limite:
            continua
        if ny < -limite or ny > limite:
            continua
        # Controlla che la posizione sia libera
        if (nx, ny) è già in stato.posizioni:
            continua
        # Calcola i nuovi contatti
        nuovi_contatti = 0
        if proteina[currStateProtLen] == 'H':
            for j da 0 a currStateProtLen-2:
                if proteina[j] == 'H' &&
                    (nx, ny) e stato.posizioni[j] sono adiacenti:
                    nuovi_contatti += 1
        # Crea il nuovo stato
        newStato.posizioni =
            stato.posizioni + [(nx, ny)]

        newStato.contatti =
            stato.contatti + nuovi_contatti

        newStato.euristica =
            (nMassimoContatti - newStato.contatti) x (protLen -
lunghezza(newStato.posizioni))

        # Inserisce nella coda nella posizione adatta
        coda.add(newStato)
    return miglior_stato, max_contatti

```